SHERLOCK – CYBER INTELLIGENCE SYSTEM

A PROJECT REPORT

Submitted by

JAIYANTH T (2116241901037)

M. S. NANTHANA SHREE (2116241901064)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

DEPARTMENT OF CYBER SECURITY





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025 | CS23533 – FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

ODD SEMESTER 2026–2027

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "SHERLOCK – Cyber Intelligence System" is the bonafide work of "Jaiyanth T (2116241901037) and M. S. Nanthana Shree (241901064)" who carried out the work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai - 602 105

SIGNATURE

Dr. Bhuvana S, Ph.D.,

SUPERVISOR

ASSISTANT PROFESSOR (SG)

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai - 602 105

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

SHERLOCK is a classical artificial intelligence project designed as an explainable, rule-based cybersecurity expert system. It accepts a user's description of a suspicious cyber incident, represents relevant observations as facts, and compares those facts with a predefined knowledge base of IF–THEN rules. The system is intended to demonstrate knowledge representation, rule matching, logical inference and explainable decision support without machine learning or trained prediction models.

The proposed workflow normalizes the supplied description or structured input, identifies indicators such as repeated failed logins or suspicious links, and checks the resulting facts against explicit rules. When a rule is satisfied, SHERLOCK returns a supported possible threat category, an evidence trace, a severity label when justified by defined criteria, and defensive recommendations. When the evidence is insufficient, the system should state that further investigation is required rather than assert a definitive finding.

The target application uses Python for rule evaluation, Flask for request handling, HTML, CSS and JavaScript for the browser interface, and JSON or structured Python definitions for the knowledge base. The inference engine is designed around rule matching and forward chaining. Visual presentation and application routes must not be confused with verified implementation: any module not confirmed in the available source code remains a proposed design.

A representative scenario is repeated failed authentication attempts within a short time window. A predefined rule can map this evidence to a possible brute-force attempt and recommend reviewing authentication logs, investigating source addresses, applying suitable access controls and enabling multi-factor authentication. MITRE ATT&CK technique T1110 is a reference mapping for brute-force behaviour, not an independent detection algorithm.

Evaluation should use controlled examples to check rule activation, explanation fidelity, severity handling, insufficient-evidence responses and input validation. The report distinguishes illustrative scenarios from observed application results. SHERLOCK is an educational first-level analysis tool; it is not an antivirus, does not continuously monitor network traffic and cannot guarantee detection of every attack or zero-day vulnerability.

Keywords: SHERLOCK, Cyber Intelligence System, Classical Artificial Intelligence, Rule-Based Expert System, Knowledge Base, IF–THEN Rules, Forward Chaining, Explainable Cybersecurity.

ACKNOWLEDGEMENT

We express our sincere thanks to our beloved and honourable chairman Mr. S.**Meganathan B.E., F.I.E., and the chairperson Dr. M. Thangam Meganathan Ph.D., for their**timely support and encouragement.We are greatly indebted to our respected and honourable principal Dr. S.N. Murugesan

M.E., Ph.D., for his able support and guidance. We express our sincere thanks to Dr. P. Tamilselvi Ph.D., Professor and Head,

Department of Computer Science and Engineering (Cyber Security). We also extend our sincere and hearty thanks to our internal guide Ms. Divya R, for her

valuable guidance and motivation during the completion of this project.We would like to extend our special gratitude to our mentor Dr. Bhuvana S, Ph.D.,

Assistant Professor (SG), for her constant guidance, valuable suggestions and continuous

encouragement throughout our project journey.Our sincere thanks to our family members, friends and other staff members of computer

science engineering.

1. JAIYANTH T (2116241901037)

2. M. S. NANTHANA SHREE (241901064)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	ACKNOWLEDGEMENT	**iv**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Objectives	2
	1.4 Scope	2
	1.5 Security Requirements and Security Considerations	3
	1.6 SDG Alignment	4
2	**LITERATURE REVIEW**	**5**
	2.1 Existing System and Rule-Driven Security Approaches	5
	2.2 Research Gaps	6
	2.3 Proposed System	7
	2.3.1 Overview of the Proposed System	7
	2.3.2 Objectives of the Proposed System	7
	2.3.3 Advantages of Proposed System	7

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
3	**SYSTEM DESIGN**	**8**
	3.1 System Architecture	8
	3.2 Module Design	10
	3.3 Data Flow and UML Diagrams	13
	3.4 Threat Model and Attack Surface	19
	3.5 Security Controls	20
4	**SYSTEM DESCRIPTION**	**21**
	4.1 Knowledge Base Design	21
	4.2 Input Processing	23
	4.2.1 Algorithms and Methodology	25
	4.2.2 Illustrative Results and Discussion	28
5	**CONCLUSION**	**32**
REFERENCES		**33**
ILLUSTRATIVE INTERFACE MOCK-UPS		**34**
APPENDIX A	Knowledge Base Inventory	38
APPENDIX B	Proposed Data Dictionary	42
APPENDIX C	Inference Pseudocode	44
APPENDIX D	Implementation and Acceptance Plan	46

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Project objectives and acceptance evidence	2
1.2	SDG alignment	4
2.1	Literature and design relevance	5
2.2	Comparison of rule-driven approaches	6
3.1	Hardware requirements	9
3.2	Software requirements	9
3.3	Core module responsibilities	10
3.4	Planned knowledge and application data tables	11
3.5	Threat model	19
3.6	Security controls and verification	20
4.1	Illustrative knowledge-base inventory	21
4.2	Input-processing behavior and planned handling	23
4.3	Illustrative rule outcomes	28
4.4	Proposed rule checks	30
4.5	Planned evaluation measures	31
A.1	Knowledge-base rule inventory	38
A.2	Knowledge-base rule inventory continued	40
B.1	Knowledge data dictionary	42
B.2	Application data dictionary	43
D.1	Planned acceptance tests	46
D.2	Proposed development milestones	47

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	SHERLOCK system architecture	8
3.2	SHERLOCK incident-analysis data flow	13
3.3	SHERLOCK use-case diagram	14
3.4	SHERLOCK forward-chaining activity diagram	16
3.5	SHERLOCK analysis sequence diagram	17
3.6	SHERLOCK knowledge and result data model	18
S.1	SHERLOCK incident input workspace (illustrative mock-up)	34
S.2	SHERLOCK rule-matching result (illustrative mock-up)	35
S.3	SHERLOCK knowledge-base browser (illustrative mock-up)	36
S.4	SHERLOCK reasoning and limitations (illustrative mock-up)	37

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

SHERLOCK is an academic cybersecurity expert system that demonstrates how explicitly represented facts and IF–THEN rules can support a transparent first-level assessment of suspicious incidents. Its purpose is to make a result inspectable: a reviewer should be able to identify the supplied observations, the rule that matched, the supported conclusion and the recommendations derived from that rule. The project focuses on classical AI concepts rather than statistical learning.

The target design separates user input, a knowledge base, an inference engine, threat analysis, and explanation and recommendation. The proposed Python and Flask application can accept a short incident description or structured observations, normalize known indicators, and evaluate the facts against reviewed rules. No machine-learning, deep-learning, neural-network or large-language-model component is part of the design.

1.2 PROBLEM STATEMENT

Cybersecurity observations are often ambiguous to inexperienced users. Repeated failed logins, suspicious messages, unusual account activity and unexpected file changes may require investigation, but a user may not know what they indicate. SHERLOCK addresses the problem of translating supplied observations into a traceable, rule-supported assessment without overstating certainty.

1.2.1 Reasoning workflow and project context

A typical analysis begins when a user submits an incident description. Input processing extracts or validates known facts, the inference engine evaluates applicable rules, and the result layer presents the possible threat category, evidence and recommended next steps. Every displayed conclusion should remain linked to the facts and rule identifiers that support it.

The main engineering challenge is to keep the explanation faithful to the actual rule evaluation. If a rule condition changes, the knowledge-base version and result trace should make the change reviewable. The system should also distinguish supplied facts from inferred conclusions and should report insufficient evidence when no rule adequately supports a threat category.

The report therefore treats the prototype as a worked foundation for an educational application.

1.3 OBJECTIVES

Table 1.1 links the project objectives to observable acceptance evidence. The central objective is an explainable expert system whose outputs can be traced to supplied facts and predefined rules. Keeping domain knowledge separate from the inference procedure makes the system easier to inspect and maintain.

Table 1.1 Project objectives and acceptance evidence

Objective	Evidence required
Represent cybersecurity knowledge	Structured indicator facts, threat categories and stable rule identifiers.
Match explicit rules	Repeatable activation of IF–THEN rules for fixed input facts.
Explain each finding	Matched premises, fired-rule ID, conclusion and recommendation trace.
Apply forward chaining	Newly derived facts can activate further rules within a defined limit.
Handle insufficient evidence	No unsupported conclusion when rule premises are incomplete.
Support safe use	Input validation, clear limitations and defensive guidance.

1.4 SCOPE

The project covers requirements analysis, literature review, system design, knowledge representation, rule matching, forward-chaining workflow, explainable output and controlled illustrative scenarios. The proposed stack is Python, Flask, HTML, CSS, JavaScript and a JSON or structured Python knowledge base. Optional persistence or administration features should be treated as planned unless confirmed in the implementation.

The project does not provide antivirus protection, automated network-wide monitoring, guaranteed incident detection, zero-day detection or a substitute for professional incident response. It does not use machine learning or train predictive models. Results depend on the accuracy of the user's input and the coverage and quality of the predefined rules.

1.4.1Scope boundaries and acceptance

Acceptance should be based on repeatable software behaviour. For a fixed set of observations and a fixed knowledge-base version, the engine should activate the same rules and return the same supported conclusions. A reviewer should be able to reconstruct each result from the listed facts, rule conditions, severity criteria and recommendation mapping.

1.5 SECURITY REQUIREMENTS AND SECURITY CONSIDERATIONS

The security design must protect incident analysis confidentiality, rule integrity and service availability. Guest incident analyses should avoid persistent personal data. Stored histories and generated reports must be accessible only to their owners or an explicitly authorized role. Knowledge-base edits need separate authorization because a changed weight or rule can alter every subsequent result.

Input should be bounded and validated against known indicator identifiers where structured input is used. Rule definitions must be treated as data rather than executable expressions. If history or administration is implemented, access control and ownership checks are required. These controls are requirements for a secure web version and should not be described as implemented without evidence.

1.6 SDG ALIGNMENT

Table 1.2 SDG alignment

S. No.	SDG No. and goal	Relevant target	Project contribution and alignment
1	SDG 4 – Quality Education	Technical and vocational skills	Direct educational alignment through knowledge engineering, logical inference and secure software development.
2	SDG 9 – Industry, Innovation and Infrastructure	Resilient digital infrastructure	Indirect conceptual alignment through cybersecurity awareness; no operational security impact has been measured.

Table 1.2 maps the project to technical education and responsible digital security practice. The alignment is educational: the project provides a setting for studying knowledge engineering, inference and secure software development. It does not establish a measured reduction in cyber incidents or improved organisational security outcomes.

1.6.1 Applying the requirements

Security requirements should be attached to individual workflows. A history request needs an owner check, a knowledge edit needs an administrative permission check, and an export needs the same access decision as the result it represents. Keeping these decisions at the server prevents the interface from becoming the sole enforcement point. The educational contribution can be assessed through tasks such as tracing a fired rule, identifying missing evidence and explaining why a rule did or did not activate. Such tasks connect the project to foundational AI learning outcomes without treating the application as a substitute for a security analyst.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Classical expert systems separate domain knowledge from a reasoning mechanism. Rule-based intrusion detection and signature-driven security tools also rely on explicit patterns or conditions, although their operational scope and data sources differ. SHERLOCK applies these established ideas in a small, inspectable educational workflow rather than claiming that rule-based cybersecurity reasoning is new.

Table 2.1 Literature and design relevance

Work	Relevant contribution	Use in SHERLOCK
Russell and Norvig, AIMA (4th ed.)	Knowledge representation, logical agents and inference.	Foundational terminology for facts, rules and reasoning.
Buchanan and Shortliffe (eds.), MYCIN (1984)	Historical rule-based expert-system architecture and explanations.	Illustrates separation of knowledge from inference; original-domain conclusions are not transferred.
Giarratano and Riley (2005)	Expert-system design, rule representation and inference engines.	Supports the modular expert-system design.
Denning (1987)	Foundational intrusion-detection model.	Provides background on security-event analysis.

Table 2.1 relates the AI literature to reasoning methods and the ASVS standard to security requirements. The historical expert-system works support the design foundations; they are not current web-security standards.

2.1.1 Implications for the design

Separating the knowledge base from the inference engine allows a reviewer to inspect a rule, change it in a controlled copy and observe how the supported conclusions change. The rule identifier and version should make every change reviewable.

Explanation identifies the observations used, the rule identifier, satisfied and unsatisfied conditions, severity rationale and recommendation mapping. A persuasive paragraph generated independently of the inference result would not establish explainability. The design therefore favours a structured evidence record that can be rendered consistently in the browser and in any later report export.

The literature review guides terminology and architecture.

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Table 2.2 compares manual interpretation, signature/rule-driven tools and the proposed educational expert system. Existing security products may inspect telemetry unavailable to SHERLOCK; the project instead focuses on explainable reasoning over user-supplied facts and a small, predefined knowledge base.

Table 2.2 Comparison of the project baselines

Dimension	Manual interpretation / signature rules	SHERLOCK design
Knowledge	Depends on user expertise or product signatures.	Explicit, inspectable IF–THEN rules.
Input	May rely on system telemetry unavailable here.	User-supplied incident description and structured facts.
Reasoning	Varies by tool and rule engine.	Rule matching with forward-chaining design.
Explanation	May be product-specific.	Shows matched evidence and fired rule.
Limitations	Coverage depends on signatures, sensors and configuration.	Coverage depends on supplied facts and authored rules.

2.2 RESEARCH GAPS

The system does not yet establish broad rule coverage, automatic telemetry collection, formal verification or detection accuracy on a representative incident dataset. These are evaluation and development gaps, not evidence that the system is superior to existing security tools. Rule provenance, a bounded inference trace and systematic testing are appropriate next steps.

Priorities arising from the gaps

A first improvement is a stable representation of observations. Present, absent and unknown states should remain distinct so that unanswered questions do not silently become negative evidence. Each rule should state which facts are required and which context is necessary.

A later extension may add a formal rule agenda and multi-step proof records. Testing that extension separately from the basic rule matcher would make its contribution assessable. Security and usability checks should likewise use explicit scenarios rather than assuming that more rules guarantee better coverage.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system is a modular web application with an inference service. It accepts validated observations, reads a reviewed knowledge-base version and returns supported findings with structured explanations. Flask coordinates requests, while the browser presents the input and result. Persistence is optional and remains proposed unless implemented.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The proposed system accepts incident observations, checks them against a versioned knowledge base and returns conclusions with their supporting evidence. The five logical modules provide clear interfaces so that input handling, knowledge storage, inference, threat analysis and explanation can be reviewed independently.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

Separating the layers allows the interface and knowledge representation to evolve without rewriting the inference procedure. Structured explanations make rule defects easier to locate, and explicit missing evidence helps users understand why a conclusion was not reached. These are design benefits that require testing in the completed application.

The design offers traceability, predictable behaviour and maintainability because conclusions are derived from visible conditions rather than an opaque trained model. These are intended advantages; actual usability and effectiveness would need to be evaluated with defined test cases and users.

2.3.3.1 Proposed incident analysis experience

The interaction should begin with a short incident description and, where appropriate, structured questions about the observed activity. Before inference, the application validates the input and identifies unknown or conflicting facts. The result view then presents matched rules, evidence, severity rationale and practical defensive actions.

Follow-up questions should be optional, focused and bounded. A response updates the current observation set and triggers a fresh calculation against the same knowledge snapshot. If the user decides to save, the application stores the resulting evidence and version together. The export then presents that same snapshot, so differences between an on-screen result and a downloaded report can be detected during testing.

This workflow turns explainability into a visible part of the incident analysis rather than a separate summary appended after scoring.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The target architecture uses a browser presentation layer, a Flask request layer, an inference engine and a knowledge-base repository. The boundary between the browser and server is untrusted. The inference engine receives validated facts rather than executing raw user text. Any persistent storage layer is optional and must be verified against the implementation.

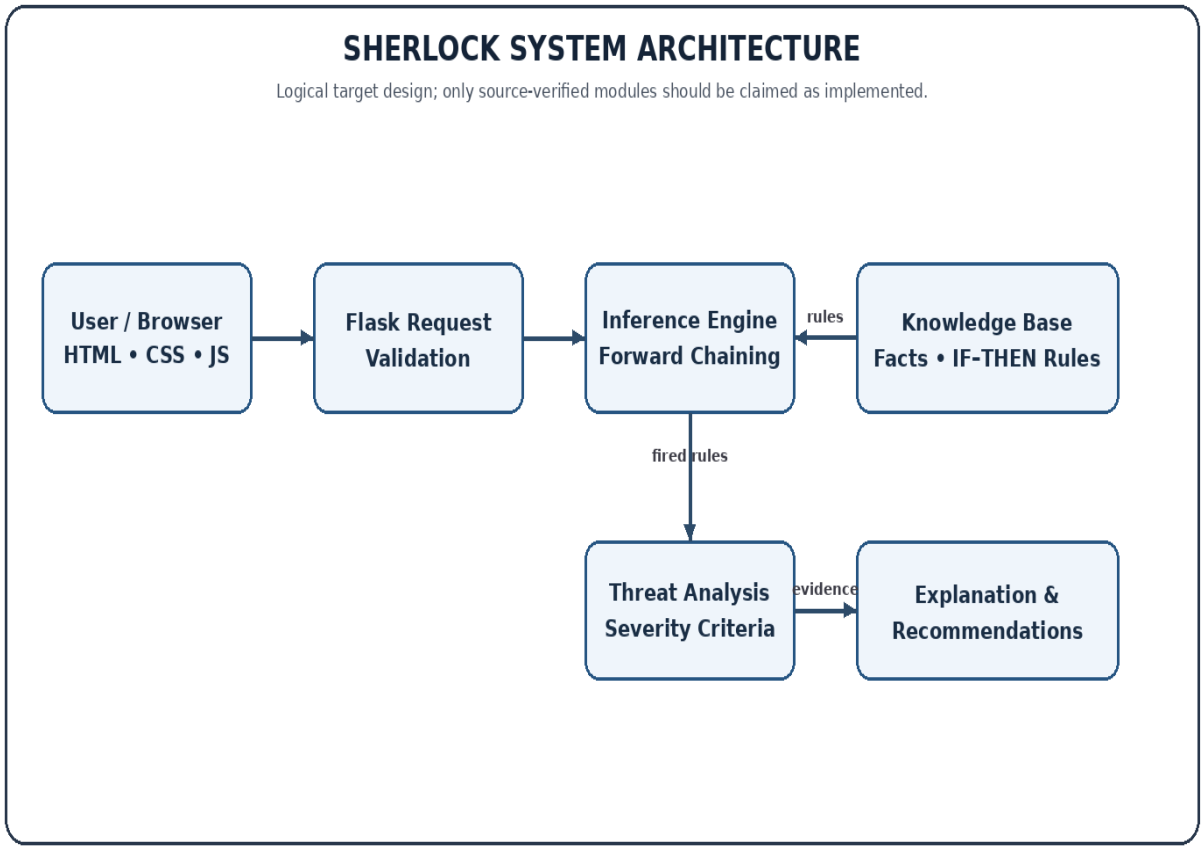


Figure 3.1 SHERLOCK system architecture

Figure 3.1 shows the target architecture: user input is validated, facts are evaluated against the rule base, and the resulting evidence is used to prepare a threat assessment and recommendations. The diagram represents the proposed modular design, not a claim that every component has been implemented.

3.1.1 Layer interaction

Each layer should expose a narrow contract: validated observations enter the inference engine, and structured evidence leaves it. Knowledge access belongs in a controlled repository, while request validation belongs in the application workflow. This division makes failures easier to isolate and allows one reasoning result to be presented in different formats.

3.1 SYSTEM ARCHITECTURE

Table 3.1 Hardware requirements

Component	Proposed development configuration
Processor	General-purpose dual-core processor or better.
Memory	4 GB RAM minimum planning allowance; 8 GB preferred for development tools.
Storage	At least 1 GB free for source, dependencies and demonstration outputs.
Graphics	No dedicated GPU required by the rule-based design.
Network	Local development can use loopback; hosting requires a secured network connection.

Table 3.2 Software requirements

Component	Role
Python	Core application logic and explicit rule evaluation.
Flask	HTTP routes and server-side request workflow.
HTML, CSS and JavaScript	Browser forms, results and user interaction.
JSON / structured Python definitions	Stores facts, rule premises, conclusions, severity criteria and recommendations.
Visual Studio Code	Development, editing and debugging environment.
Git (optional)	Version control and change review.
SQLite (optional / proposed)	Persistent storage only if history or administration is implemented.

Table 3.1 lists planned hardware allowances; Table 3.2 identifies the software components. These are planning assumptions, not measured minimum requirements. Dependencies must be pinned and tested; inference needs no external AI API or model-training service.

3.1.2 Development and deployment preparation

A reproducible setup should record the interpreter version, installed dependencies and database initialization steps. A small demonstration knowledge snapshot and fixed sample inputs should accompany the source so that another student can verify the calculation before enabling account or history features.

3.2 MODULE DESIGN

Table 3.3 defines the five core AI modules, their inputs and their outputs. Their interfaces should return structured data, allowing the command-line demonstration and web application to share the same inference engine.

Table 3.3 Core module responsibilities

Module	Input	Output
1. User Input	Incident description and structured observations.	Validated facts and input warnings.
2. Knowledge Base	Reviewed threat categories, indicators and IF–THEN rules.	Versioned rule set.
3. Inference Engine	Validated facts and knowledge base.	Fired rules and derived facts.
4. Threat Analysis	Fired-rule trace and severity criteria.	Supported possible category and severity, or inconclusive status.
5. Explanation and Recommendation	Evidence trace and conclusion.	Readable reasoning and defensive actions.

A result record should carry the fired rule identifier, supplied facts, matched and missing premises, derived conclusion, severity rationale, recommendations and knowledge version. This contract makes the result independently reviewable. A rule identifier alone does not demonstrate that a formal multi-step proof was executed.

The explanation view should present the evidence for each supported finding, not only the first result. Where more than one rule matches, the interface should show the relevant rule traces or explain the documented conflict-resolution policy.

3.2.1 Module contracts and error handling

Validation errors should be returned as structured messages that identify the affected input without exposing internal exceptions. The inference module should receive an already validated knowledge snapshot, but it should still reject an invalid rule premise schema rather than return a misleading value.

The explanation module should consume the structured inference result directly. It should not independently rerun the rule logic, because duplicated reasoning implementations can diverge after a maintenance change. A shared result record makes it possible to compare the browser output, any saved record and any later export.

Clear contracts allow each module to be reviewed with small controlled examples before the full application is assembled.

3.2.2 Module Design

Module 1: User Input Module

Collects incident descriptions and structured observations, validates the input, and identifies input warnings.

Module 2: Knowledge Base Module

Stores and manages cybersecurity threat categories, indicators, and IF–THEN rules for incident analysis.

Module 3: Inference Engine Module

Applies forward-chaining techniques to evaluate validated facts against knowledge-base rules and derive new facts.

Module 4: Threat Analysis Module

Evaluates matched rules and severity criteria to identify possible threats or return an inconclusive result when evidence is insufficient.

Module 5: Explanation and Recommendation Module

Presents the reasoning behind each finding, supporting evidence, and appropriate defensive actions.

Module 6: User Authentication Module

Manages user registration, login, and access to the SHERLOCK AI application, where authentication is implemented.

Module 7: Result Visualization and Reporting Module

Displays incident analysis results, threat severity, supporting evidence, and recommendations in a clear report format.

Module 8: Knowledge Base Administration Module

Manages cybersecurity rules, indicators, threat categories, and knowledge-base updates to maintain accurate and consistent analysis.

3.2 MODULE DESIGN

The supporting modules provide the planned application workflow. Authentication manages registered and guest access. Consultation routes pass validated observations to the engine. History and report modules use the same result snapshot. Administrative routes manage knowledge separately from personal incident analysis data.

Table 3.4 Planned database tables

Table	Purpose
knowledge_rules	Rule identifier, premises, conclusion, severity criteria and recommendation mapping.
indicator_catalog	Canonical indicator IDs, display labels and reviewed aliases.
threat_categories	Threat category names and descriptions.
rule_versions	Published knowledge snapshots and change metadata.
analysis_history (optional)	User-owned analysis snapshot, if persistence is implemented.
analysis_facts (optional)	Facts associated with a saved analysis.
users (optional)	Account identity and password hash if authentication is implemented.
audit_events (optional)	Administrative changes and security-relevant actions.
recommendations	Reviewed defensive actions linked to rule conclusions.
severity_criteria	Documented criteria for assigning severity.
rule_premises	Structured conditions associated with a rule.

Table 3.4 organizes the eleven planned tables for knowledge, accounts and incident analysis history. Validation, review and snapshot fields support the design without implying that the database is already implemented. Appendix B defines proposed constraints.

3.2.1 Persistence workflow

Knowledge maintenance and incident analysis storage have different lifecycles. A reviewed knowledge change creates a published version for later incident analyses. Saving an incident analysis
records the observations and complete result produced by the version already in use. These operations should not share an implicit update path.

The repository should group related writes into a transaction so that a history record cannot be presented as complete when its observations have not been stored. A failed write should leave the user with a clear outcome and preserve the current displayed result for a deliberate retry.

3.3 DATA FLOW AND UML DIAGRAMS

The data flow separates incident analysis evidence from administrative knowledge changes. A registered user may explicitly save an incident analysis. Guest requests are processed without persistent history by default.

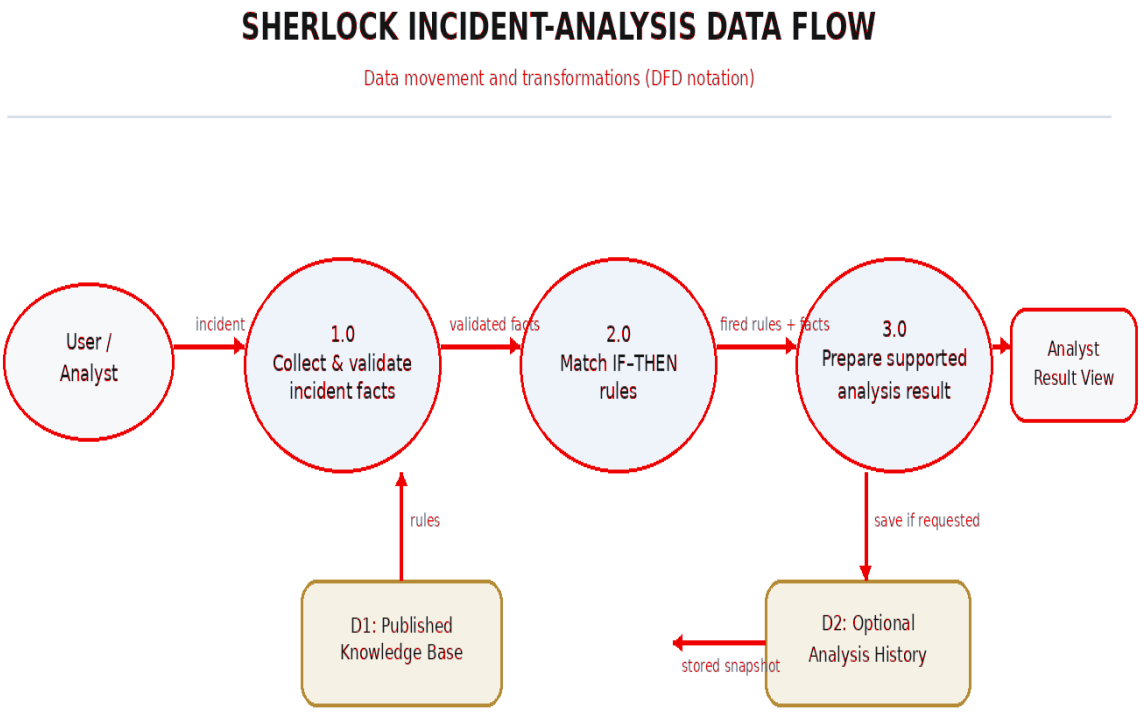


Figure 3.2 SHERLOCK incident-analysis data flow

Figure 3.2 traces observations through collection, inference and result display using circular nodes. Published knowledge versions prevent an administrative edit from changing an in-progress incident analysis. Optional saving and export use the current or stored result snapshot and require the same ownership checks as the results page.

3.3.1 Reading the data flow

The numbered processes describe transformations of data rather than individual pages. Input processing produces canonical observations; rule evaluation produces fired rules and derived facts; presentation converts the evidence trace into a readable result. Any history write is a separate optional branch.

3.3 DATA FLOW AND UML DIAGRAMS

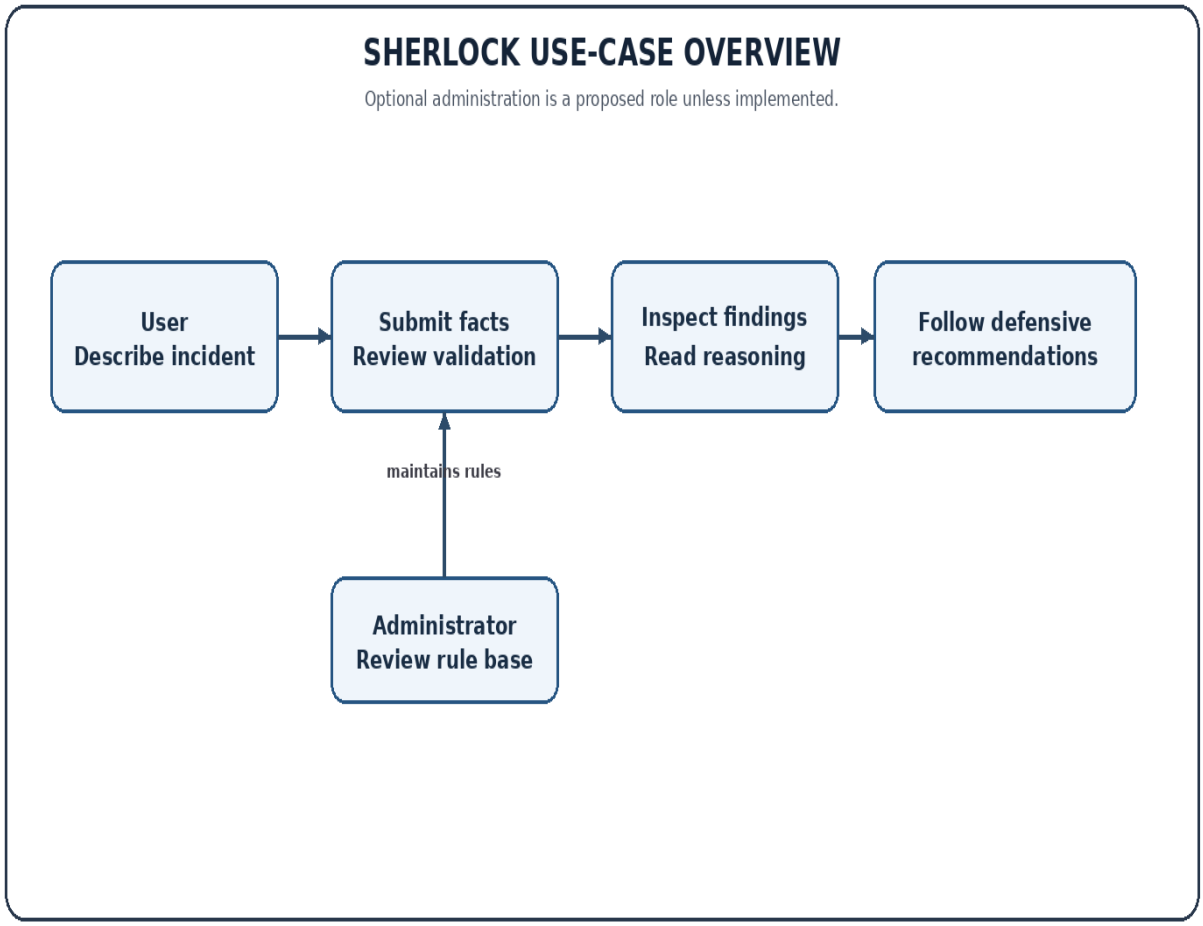


Figure 3.3 SHERLOCK use-case diagram

Figure 3.3 shows that guests can submit temporary incident analyses and inspect explanations. Registered users can also save, view and delete their own history. Administrators maintain knowledge records, but that role does not automatically permit access to every personal incident analysis.

Each history or report request must be authorized at the server. Hiding a navigation link is insufficient because a user could directly request a resource identifier. The user interface should explain when data will be saved and provide a clear way to end the incident analysis.

3.3.2 Actor responsibilities

Permissions should be expressed in terms of actions and resources. Viewing an explanation for the current temporary incident analysis does not require the same permission as retrieving an older saved report. A registered account can manage its own stored incident analyses, while knowledge publication belongs to the administrative workflow.

The use case boundary helps identify acceptance scenarios.

3.3 DATA FLOW AND UML DIAGRAMS

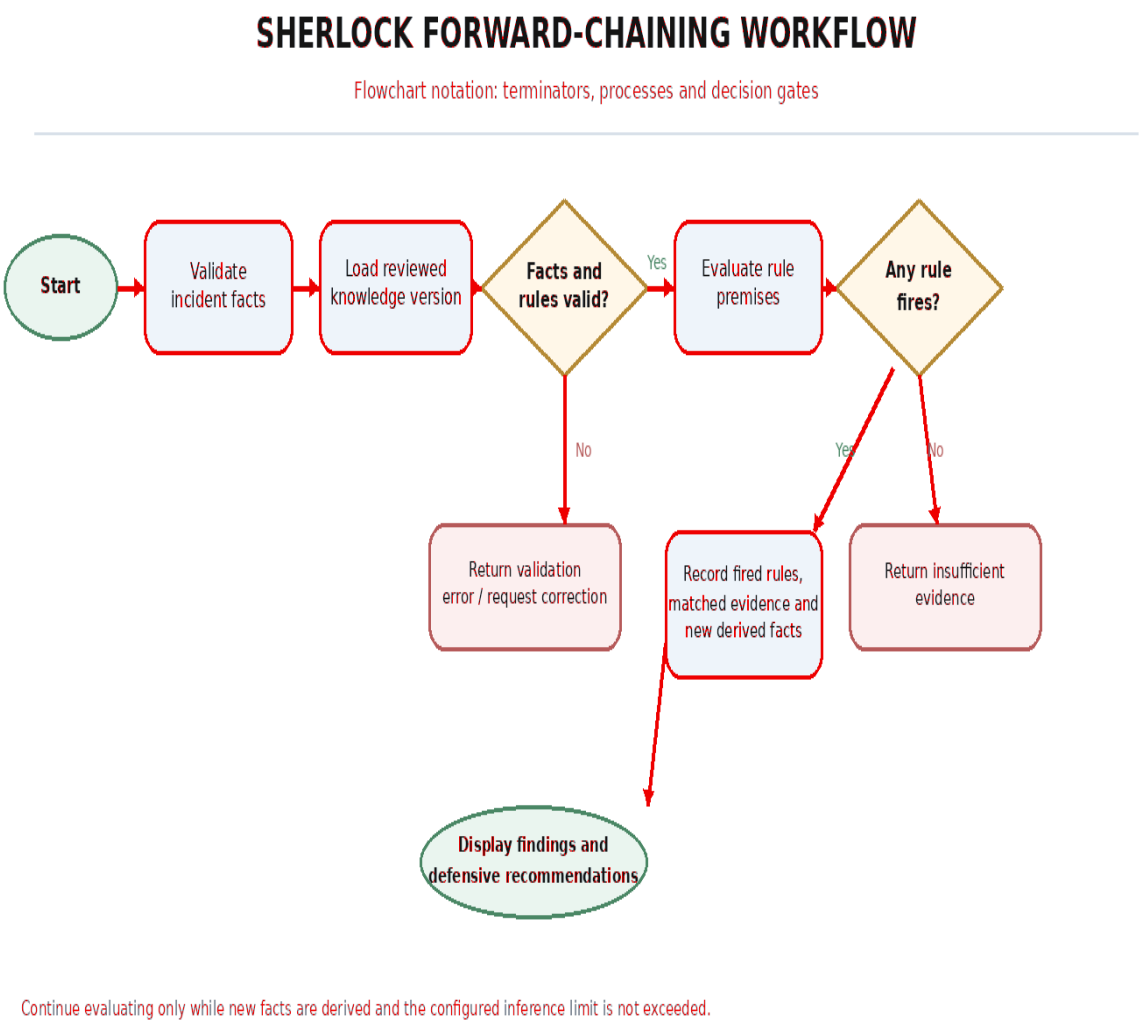


Figure 3.4 SHERLOCK forward-chaining activity diagram

Figure 3.4 shows the proposed workflow returning invalid or conflicting observations to the input stage. Unknown answers remain unknown rather than being treated as absent facts. The correction gate is a design requirement and should not be described as implemented without supporting source code.

3.3.4 Decision points and termination

The first decision checks whether the observations are suitable for evaluation. A correction response should identify the specific conflicting or unknown identifier and retain the other valid selections. This avoids forcing the user to restart the incident analysis after a recoverable input problem.

The second decision distinguishes a ranked result from a no-match outcome. Both are valid end states of the calculation and should explain what happened.

3.3 DATA FLOW AND UML DIAGRAMS

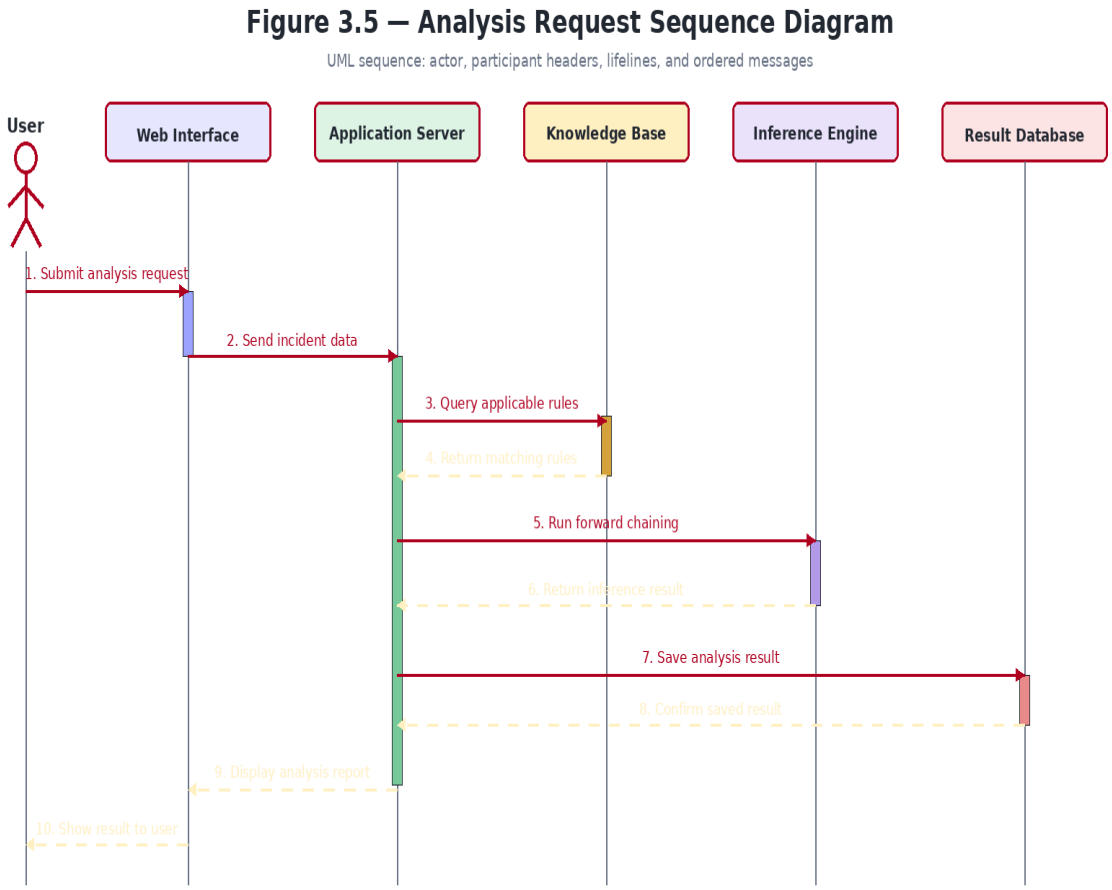


Figure 3.5 SHERLOCK analysis sequence diagram

Figure 3.5 shows the target request sequence: the controller validates input, the knowledge repository supplies a rule snapshot, and the inference engine returns findings with evidence. Any save action must follow explicit authorization and storage checks. This is a design specification, not evidence that history routes exist.

The sequence is a design specification for the web version. It does not imply that the authentication, persistence or export routes have been implemented.

3.3.5 Consistency across the request

The knowledge read should return the rule set and its version identifier. The engine should use that same snapshot throughout a request, even if an administrator publishes a newer version before the response is displayed. This supports reproducible reasoning.

A save response should identify the stored incident analysis only after the transaction succeeds.

3.3 DATA FLOW AND UML DIAGRAMS

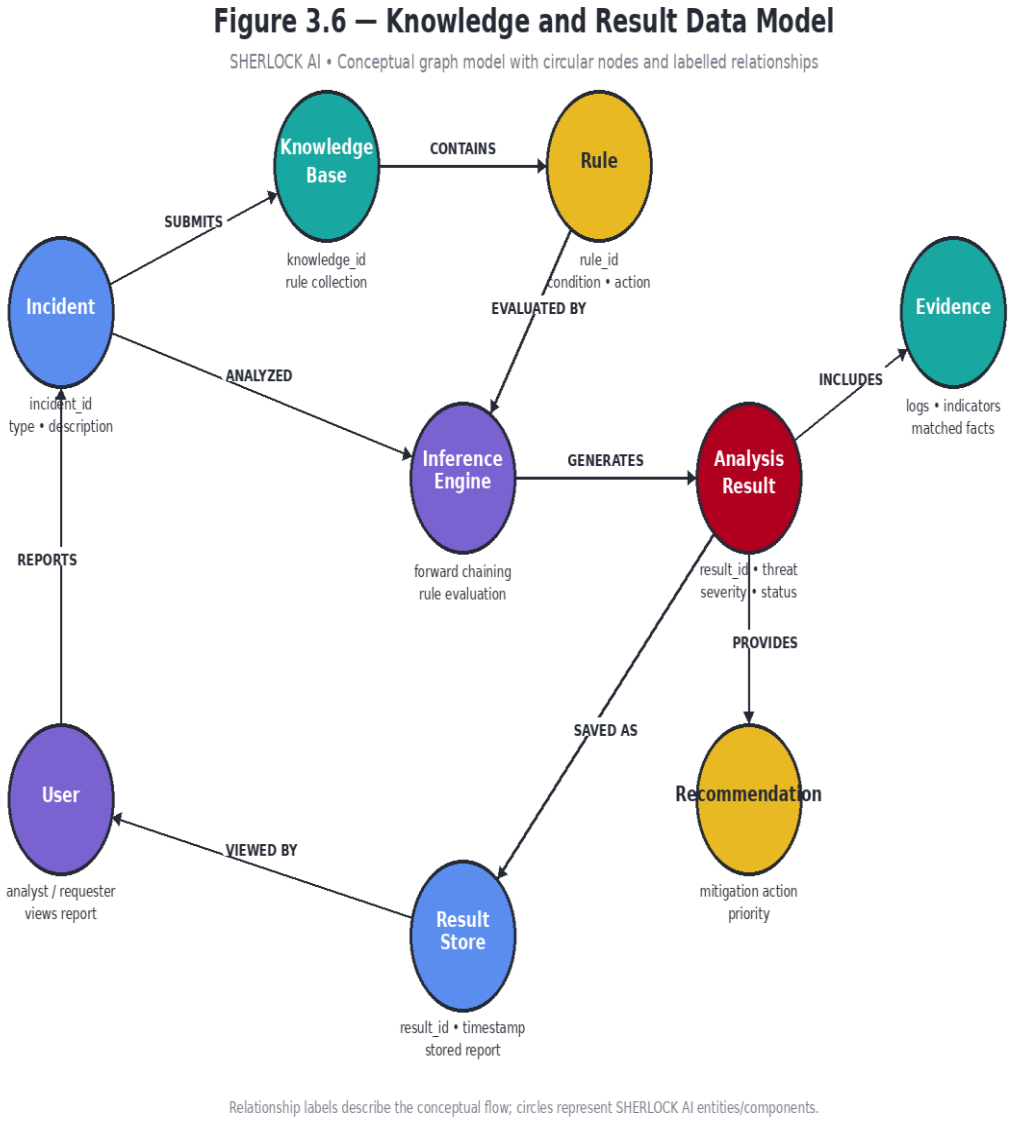


Figure 3.6 SHERLOCK knowledge and result data model

Figure 3.6 shows the conceptual relationship between the indicator catalogue, rule premises, conclusions, recommendations and optional analysis history. A stored result should retain its original evidence trace so that later rule changes do not silently alter the interpretation of an earlier result.

Foreign keys, uniqueness constraints and reviewed publication states protect the knowledge structure. The diagram summarizes the eleven logical tables; Appendix B specifies proposed field and validation choices.

The junction tables should reject duplicate links. History ownership and knowledge-version information must remain available when interpreting a saved result.

3.4 THREAT MODEL AND ATTACK SURFACE

Assets include indicator observations, account credentials, incident analysis reports, rule weights and administrative privileges. Relevant actors include an ordinary user, an unauthorized visitor, a compromised account and an administrator making an accidental error. The browser, upload interface, report endpoint and editable knowledge base form the main attack surfaces.

Table 3.5 Threat model

Threat	Affected asset or boundary	Proposed response
Rule tampering	Knowledge base and inference integrity.	Restrict edits, validate schema and version published rules.
Input manipulation	User input and indicator parsing.	Bound request size and validate known fields.
Information disclosure	Optional saved analyses or reports.	Enforce ownership checks and non-public storage.
Injection	Database queries and rendered text.	Parameterized SQL and context-appropriate output escaping.
Denial of service	Large inputs or repeated inference requests.	Request limits, bounded rule evaluation and rate controls.
Misleading result	Incomplete facts or overbroad rules.	Show evidence, limitations and insufficient-evidence outcomes.

Table 3.5 maps account spoofing, rule tampering, disclosure, injection, denial of service and untraceable edits to responses in the proposed web system. Python-file modification and terminal disclosure are more immediate local-prototype risks. Operational misuse remains an output-design risk addressed by the limits in Chapter 4.

3.4.1 Reviewing an attack path

A useful review follows a request from entry to storage and back to output. For example, a report identifier supplied by the browser must be checked against the authenticated owner before any snapshot is read or rendered. Knowledge publication similarly needs authorization before a validated draft becomes active.

The same exercise should cover failure cases. An oversized request should terminate before expensive work, and a rejected administrative change should leave the published version intact. Error messages should help the user recover without revealing another account, a private report path or unnecessary application internals.

These scenarios convert the threat model into concrete checks that can be repeated against an implemented build.

3.5 SECURITY CONTROLS

Table 3.6 Security controls and verification

Control	Implementation requirement	Verification evidence
Input validation	Validate fields and bounded observation counts.	Malformed and oversized requests are rejected.
Rule integrity	Treat rule definitions as data and validate before use.	Invalid rules cannot be published or executed.
Authentication (if present)	Hash passwords and protect sessions.	Inspect stored credentials and test session transitions.
Authorization (if present)	Enforce ownership on saved analyses and exports.	A second account cannot read another account’s records.
Data protection	Use HTTPS when hosted; keep private records outside public assets.	Check transport settings and direct resource access.
Logging	Avoid recording secrets or unnecessary incident content.	Review logs for sensitive data and access restrictions.

Table 3.6 links security controls to implementation requirements and verification evidence. These are proposed ASVS-informed checks, not a compliance claim. State-changing requests also need CSRF protection, output must be escaped, and development debugging must be disabled when hosted.

For privacy, the default guest path should avoid saving names, ages and indicator histories. Where registered users choose storage, the application needs an explicit retention policy and deletion behavior. Backup access must be restricted, and restoration must be tested before backup reliability is claimed.

3.4.2 Verification and maintenance

Controls should be checked at the boundary where they are enforced. An ownership test should request another account's history directly; an input test should submit a malformed identifier to the server; a publication test should attempt to activate an invalid card. A successful browser interaction alone does not exercise these denied paths.

The project should retain a compact verification record containing the scenario, expected behavior, actual behavior and application version. When a route or data model changes, the affected scenarios can be rerun. This makes the security requirements maintainable and keeps untested design intentions separate from demonstrated controls.

Log entries should be limited to the information needed to investigate or audit the event, with incident analysis content excluded unless a justified and authorized workflow requires it.

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The initial knowledge base should be manually authored and reviewed, not learned from a training dataset. It should contain threat categories, observable indicators, IF–THEN rules, severity criteria, explanations and defensive recommendations. The final inventory must reflect the actual rule definitions in the submitted source; this report uses illustrative rule examples where source evidence is unavailable.

Table 4.1 Knowledge-base inventory

Category	Illustrative rule examples	Purpose
Credential Access	Repeated failed logins; password spraying indicators.	Identify suspicious authentication patterns.
Phishing	Suspicious link; sender spoofing; urgent credential request.	Flag potentially deceptive messages.
Malware Indicators	Unexpected executable; suspicious file change.	Prompt defensive investigation.
Account Misuse	Unexpected login location or unusual account activity.	Recommend account review.
Network Anomaly	Unexpected outbound connection reported by user.	Prompt log review; no automatic monitoring implied.
Social Engineering	Request for credentials or one-time code.	Warn against sharing secrets.
Data Exposure	Unexpected public sharing or unauthorized transfer.	Recommend access review and containment.
TOTAL	Illustrative categories only	Not a verified count of implemented rules.

Table 4.1 provides an illustrative knowledge-base inventory by cybersecurity category. It is a proposed organization for rule authoring, not a claim that every listed rule has already been implemented or validated. The number and coverage of active rules should be verified against the actual project files.

4.1.1 Knowledge review procedure

A knowledge review record should identify the rule, reviewer, version and reason for a change. Structural checks can confirm that identifiers and premises are well-formed, but they cannot establish that a rule is complete or appropriate for every real-world environment.

Before adding a rule, the development team should resolve duplicate identifiers, document reviewed aliases and keep incomplete rules inactive. The number of rules is not a measure of their quality or coverage.

4.2 DATA PREPROCESSING

Input processing should trim unnecessary whitespace, normalize case and map recognized phrases to stable indicator identifiers. For example, “many failed login attempts” may map to repeated_failed_logins only when the mapping is explicitly defined. Unknown phrases should remain unclassified or prompt the user for clarification rather than silently becoming a known fact.

Table 4.2 Preprocessing behavior and planned changes

Input issue	Safe handling	Design note
Case and spacing	Normalize case and whitespace.	Normalization does not establish meaning.
Duplicate observations	Deduplicate identical canonical facts.	Duplicates should not increase evidence.
Unknown phrase	Ask for clarification or preserve as unknown.	Do not silently map unknown text to a threat fact.
Related terms	Use reviewed aliases only.	Do not infer equivalence automatically.
Conflicting facts	Flag the conflict and request correction.	Keep time window and account context explicit.

Table 4.2 distinguishes safe normalization from interpretation. Normalization can standardize spelling and identifiers, but it must not invent facts. Duplicate observations should not create extra evidence, and an unknown observation should remain unknown until the user supplies additional information.

Cybersecurity indicators also need context. A failed login can be benign or malicious depending on frequency, timing, account context and source. A single observation should not automatically establish an attack; rule conditions should specify any threshold or context required before a conclusion is returned.

4.2.1Normalization examples and boundaries

Normalization should produce a canonical identifier rather than a new interpretation of the user's statement. A reviewed alias can map to an existing code, while an unrecognized term should be returned for correction. The original display label can remain readable without becoming a second identifier.

4.2.2 ALGORITHMS AND METHODOLOGY

4.2.2.1Knowledge representation

Represent each recognized observation as a fact, for example FailedLoginCount(account, 12, window_5_minutes) or SuspiciousLink(message_17). A rule contains explicit conditions and a conclusion. The inference engine evaluates whether the supplied facts satisfy the conditions; it does not learn weights or discover new rules from data.

```
IF–THEN rule
IF failed_login_attempts >= threshold
AND time_window <= configured_window
THEN possible_threat = brute_force_attempt
```

A conclusion means that the supplied evidence satisfies a predefined rule, not that an attack has been conclusively proven. Rule matching and logical inference should be distinguished from confidence estimates. Any severity label must be justified by documented criteria and available evidence.

4.2.2.2Forward and backward reasoning

Forward chaining starts with known facts and repeatedly evaluates rules whose premises may be satisfied, adding supported conclusions until no additional rule fires or a defined stopping condition is reached. SHERLOCK is intended to use this approach as its primary inference model. The report must not claim a multi-step engine is implemented unless source code confirms it.

The system is deterministic for a fixed input and fixed knowledge base when rule ordering and conflict handling are defined. No weights are learned from data, and no predictive model is trained.

4.2.2.4 Evidence representation and traceability

A fact record should include a canonical indicator, its source (user-supplied or derived), and any relevant time or count information. The result trace should record which premises matched, which were missing, which rule fired and which conclusion was derived.

A formal multi-step trace should include the rule identifier, its premises, the facts that satisfied each premise, the conclusion added and the inference step number. Repeated derivations should not create an endless loop; a defined stopping condition and cycle handling are required.

4.2.2.5 Rule matching and forward chaining

For a rule R, let P(R) be its required premises and F the supplied facts. A rule fires only when every required premise is satisfied. If the premises are incomplete, the engine should report the missing evidence and refrain from presenting the rule's conclusion as established.

> RuleFires(R) = TRUE when every premise in P(R) matches the known facts
>
> Otherwise: RuleFires(R) = FALSE
>
> If no applicable rule fires, return "Insufficient evidence for a supported threat classification."

The inference engine should evaluate rules in a defined order or use a bounded agenda. Derived facts may activate additional rules, allowing forward chaining across more than one step. Only conclusions supported by fired rules should be displayed.

Severity should be assigned using documented criteria such as impact, scope, confidence in the observed facts and urgency. A high severity label must not be assigned solely because a threat name sounds serious. If the available evidence does not satisfy the threshold, the output should remain suspicious or inconclusive.

The interface should explain that a rule match is not a guarantee of attack detection. Results must identify the evidence used and the limitations of the inference, and must not imply continuous monitoring or universal attack coverage.

4.2.2.6 Interpreting a rule result

The conclusion is determined by rule conditions rather than by a percentage that resembles a prediction score. Different rules may overlap, so the output should show all relevant fired rules or explain how competing conclusions are handled.

The result record should explicitly list satisfied premises, missing premises, the rule identifier and any severity criteria applied. It should not invent evidence to complete an incomplete rule.

When several rules match, the system should apply a documented ordering or show multiple supported findings. Ordering alone is not evidence that one finding is more accurate. The user should be able to inspect the basis for each conclusion.

4.2.2.7 Backward verification and consistency checking

If a rule requires evidence that was not supplied, the system should identify that missing evidence and request clarification when appropriate. This is a premise check, not a claim that a complete backward-chaining prover has been implemented.

A future follow-up workflow can ask one bounded question at a time, accept yes, no or unknown responses, and rerun inference after each response. It should stop when the user declines, the evidence is adequate for a rule, or a configured question limit is reached.

4.2.2.8 Resolution and contradiction handling

Contradictory or ambiguous observations should be handled explicitly. For example, a report may contain both “no failed logins” and “repeated failed logins” for the same account and time window. The system should ask for correction or preserve the conflict in the explanation instead of silently selecting one fact.

Illustrative rule: A AND B IMPLIES C
Clause form: NOT A OR NOT B OR C
Facts: A, B
Negated query: NOT C

Resolve with A: NOT B OR C
Resolve with B: C
Resolve with NOT C: empty clause

This is a small logical illustration of entailment, not evidence that a formal resolution prover is implemented in SHERLOCK. The primary project workflow remains explicit IF–THEN rules and forward chaining.

4.2.2.9 Unknown evidence and bounded reasoning

A missing requirement and an explicitly absent requirement are different observations. A focused follow-up question should preserve that distinction and allow the user to decline. Recalculation should use the updated observation set while retaining the same knowledge version for the incident analysis.

A formal resolution exercise should retain each derived clause and its parent clauses so that an examiner can follow the refutation. Repeated clauses should be detected, and a resource limit should terminate the demonstration predictably. These requirements concern the proposed teaching module, which remains separate from pairwise conflict checks in the prototype.

4.3 RESULTS AND DISCUSSION

4.3.1 Worked scoring examples

The following scenario is illustrative and demonstrates the expected rule-based workflow. It is not a verified execution log or a measured experimental result. The rule thresholds, severity mapping and exact output must be checked against the implemented knowledge base before being reported as observed behaviour.

Table 4.3 Illustrative rule outcomes

Input and rule	Matched premises / required premises	Penalty / status	Illustrative outcome
12 failed logins in 5 minutes; threshold 10	All premises satisfied	Rule fires	Possible brute-force attempt
“Several failed logins”; no count or time	Threshold premise missing	No fire	Insufficient evidence for threshold rule
Suspicious link only	No credential submission fact	Partial evidence	Potential phishing indicator; investigate
Empty input	No facts supplied	No rule fires	Insufficient evidence

For example, if the configured rule requires at least ten failed logins within five minutes and the user supplies twelve attempts within that window, the rule premises are satisfied. If the user supplies only “some failed logins” without count or time context, the system should not fire that threshold-specific rule.

The example illustrates why evidence and rule conditions must be shown together. SHERLOCK should identify the matched facts, state that the result is a possible brute-force attempt, and recommend further investigation rather than claiming certainty.

4.3.2 Reproducing a worked example

A reproducible test case should retain the exact input facts, knowledge-base version, expected rule activation, expected conclusion and expected recommendations. A reviewer can then check whether the inference result follows from the written conditions.

4.3.3 Deterministic behavior checks

Table 4.4 Proposed rule checks

Case	Observation	Interpretation
Threshold met	12 failed logins within configured 5-minute window.	Brute-force rule may fire if threshold is configured as 10.
Missing context	“Many failed logins” without a count or window.	Do not claim threshold rule fired.
Suspicious link	User reports an unexpected message link.	Recommend not clicking; verify sender through trusted channel.
Unknown input	Text does not map to a known indicator.	Request clarification or report no supported rule.
Conflicting facts	Both repeated and no failed logins for same window.	Request correction.
Empty input	No description or structured facts.	Return insufficient evidence.
Multiple rules	More than one rule matches.	Show each supported rule or documented priority.
Rule boundary	Evidence does not meet threshold.	Do not force a definitive category.

The test table should cover a satisfied rule, incomplete premises, unknown input, contradictory facts, duplicate observations and empty input. These are proposed acceptance scenarios unless actual test logs are supplied.

The no-match path returns before the final academic disclaimer in the listing. The planned application should place its educational-use notice consistently on every result state, including errors and empty results.

4.2.3.4 Extending the behavior checks

Additional controlled cases should cover multiple applicable rules, incomplete premises and a malformed rule rejected during loading. The tests should check the displayed evidence and recommendation, not merely whether a result is returned.

A useful test record includes the complete input and expected evidence, not only the first conclusion. This detects regressions in premise lists and explanations even when the same threat category remains visible.

4.3.5 Evaluation plan and limitations

Table 4.5 defines proposed checks for functional correctness, explanation fidelity, robustness, performance, security and usability. Results should be recorded only after running the identified application build; no performance or detection-accuracy measurements are asserted in this report.

Table 4.5 Planned evaluation measures

Measure	How it will be evaluated
Functional correctness	Expected rule activation, conclusion and recommendation for controlled cases.
Explanation fidelity	Every displayed premise and conclusion matches the rule trace.
Robustness	Unknown identifiers, duplicates, conflicts, empty and oversized requests.
Performance	Record configuration, knowledge size, repetitions and latency if measured.
Security	Input validation, rule integrity, access control and report access where implemented.

Detection precision, recall and false-positive rates require a representative, independently labelled dataset and a defined evaluation procedure. No such dataset or measured results are supplied here, so the report does not claim quantified detection accuracy.

Limitations include a finite hand-authored rule set, incomplete indicator vocabulary, dependence on user-supplied facts, possible false positives and false negatives, limited temporal context and inability to detect behaviour not represented in the rules. SHERLOCK does not replace professional incident response or endpoint/network security products.

4.3.6 Evaluation records and reporting

Each comparison should retain its knowledge version, fixed case set and expected outputs. Performance measurements should state whether they cover only the inference function or the entire request, because validation, storage and PDF generation introduce separate work.

Usability tasks should ask participants to interpret a rule trace, identify missing evidence and recognize an inconclusive result. Findings from these tasks can guide wording and layout changes. They should be reported as evidence about interaction with the educational system.

CHAPTER 5

CONCLUSION

SHERLOCK demonstrates the foundational AI concepts of knowledge representation, explicit IF–THEN rules, rule matching, forward chaining and explainable decision support in a cybersecurity context. Its modular design separates user input, knowledge storage, inference, threat analysis and recommendations.

The illustrative brute-force scenario shows how repeated failed authentication attempts can satisfy a predefined rule and produce a traceable possible-threat finding. The conclusion remains conditional on the configured threshold and supplied facts; a rule match is not a verified attack or a measured accuracy result.

The next development stage should verify the knowledge schema, implement and test the inference workflow, record fired-rule traces, and add controlled tests for incomplete evidence and conflicting facts. Any web routes, storage, administrative controls or export features should be documented as implemented only after they are present in the submitted code.

Future extensions may include a larger reviewed knowledge base, richer observation context, bounded multi-step forward chaining, rule versioning, audit records and a carefully controlled incident-history feature. These enhancements should preserve explainability and should not be described as completed functionality before implementation.

5.1 Development priorities

The immediate priority is an implementation that preserves the evidence contract from input to result. Canonical observations, a fixed knowledge version and structured fired-rule records provide a common foundation for the interface and test suite.

The next priority is repeatable verification. The arithmetic examples can become controlled regression cases, while the proposed acceptance scenarios can exercise input correction, ownership, publication and export consistency. Results should be recorded only after execution against an identified build.

The academic value of SHERLOCK lies in demonstrating how explicit knowledge, logical inference and evidence-linked explanations can support an understandable first-level cybersecurity assessment.

5.2 Future Work

Enhance SHERLOCK AI by expanding its cybersecurity knowledge base, improving threat detection accuracy, and integrating real-time threat intelligence for proactive security recommendations.

REFERENCES

[1] Buchanan, B. G., & Shortliffe, E. H. (Eds.). (1984). Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project. Addison-Wesley.

[2] Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

[3] Giarratano, J. C., & Riley, G. D. (2005). Expert Systems: Principles and Programming (4th ed.). Thomson Course Technology.

[4] van Melle, W., Shortliffe, E. H., & Buchanan, B. G. (1984). EMYCIN: A knowledge engineer's tool for constructing rule-based expert systems. In Rule-Based Expert Systems (pp. 302–313). Addison-Wesley.

[5] MITRE. (n.d.). MITRE ATT&CK: Tactics, techniques, and procedures. https://attack.mitre.org/

[6] MITRE ATT&CK. (n.d.). T1110: Brute Force. https://attack.mitre.org/techniques/T1110/

[7] Denning, D. E. (1987). An Intrusion-Detection Model. IEEE Transactions on Software Engineering, SE-13(2), 222–232. https://doi.org/10.1109/TSE.1987.232894

[8] Forgy, C. L. (1982). Rete: A fast algorithm for the many pattern/many object pattern match problem. Artificial Intelligence, 19(1), 17–37. https://doi.org/10.1016/0004-3702(82)90020-0

[9] National Institute of Standards and Technology. (2023). Guide to Intrusion Detection and Prevention Systems (IDPS) (SP 800-94). https://csrc.nist.gov/pubs/sp/800/94/final

[10] OWASP Foundation. (n.d.). Application Security Verification Standard. https://owasp.org/www-project-application-security-verification-standard/

APPLICATION SCREENSHOTS

The following figures are illustrative interface concepts for the SHERLOCK workflow, not screenshots of a verified running application. They show the intended presentation of user-supplied observations, fired rules, evidence, severity rationale and defensive recommendations.

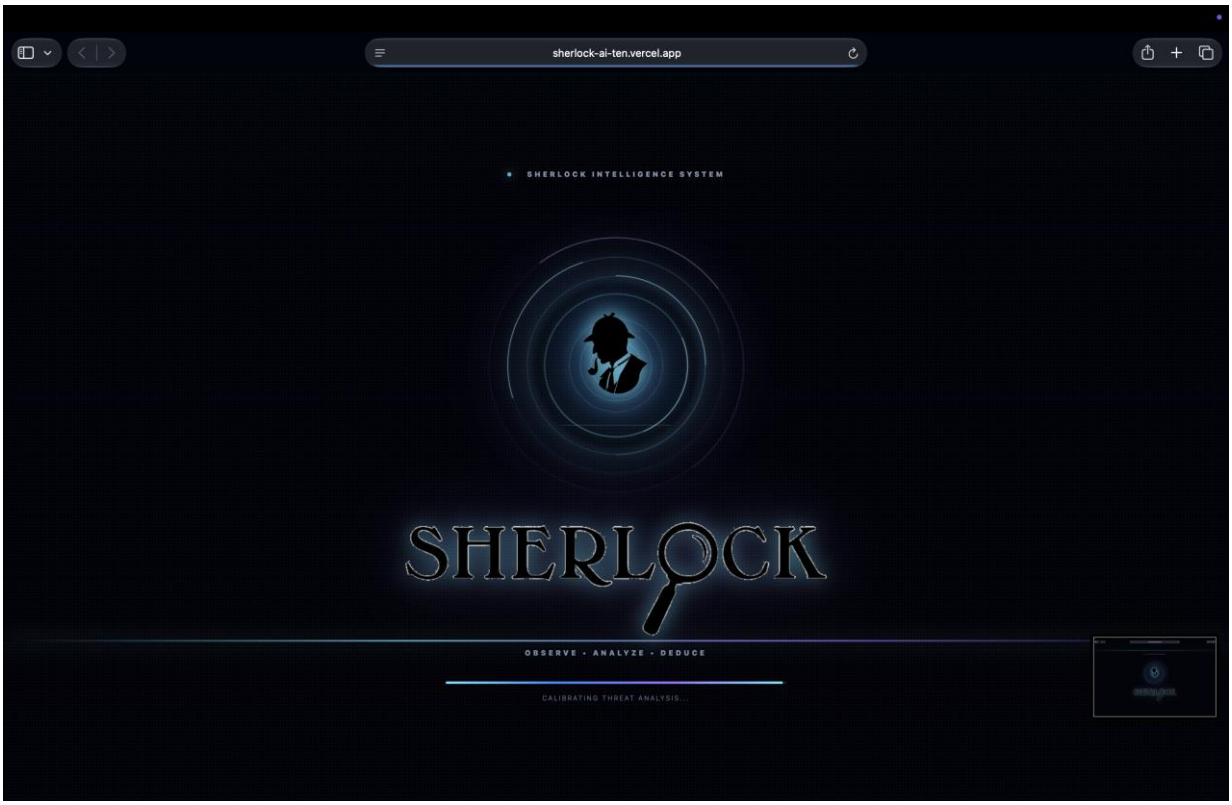


Figure S.1 SHERLOCK incident input workspace (illustrative mock-up)



This illustrative mock-up represents the loading screen of our project

APPLICATION SCREENSHOTS CONTINUED

This illustrative screen shows a sample incident input beside a possible rule-based result. The layout is intended to help users compare the supplied observations with the evidence supporting a finding; it is not a real execution screenshot.

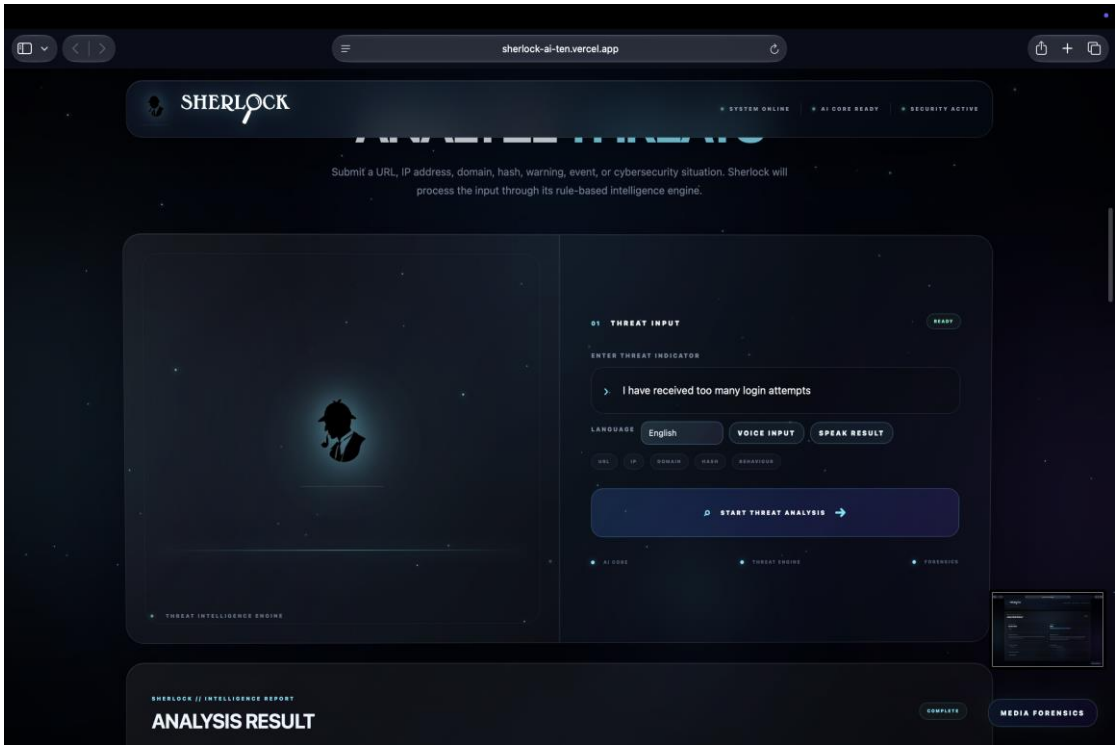


Figure S.2 SHERLOCK rule-matching result (illustrative mock-up)

This illustrative result view presents a possible brute-force finding, the repeated-failed-login evidence, the matching IF–THEN rule and defensive recommendations. It is a design example rather than an observed application output.

APPLICATION SCREENSHOTS CONTINUED

The knowledge-base view provides a separate place to inspect the demonstration rules. Search and category labels support browsing, while each threat profile offers access to the rule conditions and indicator weights used by the interface.

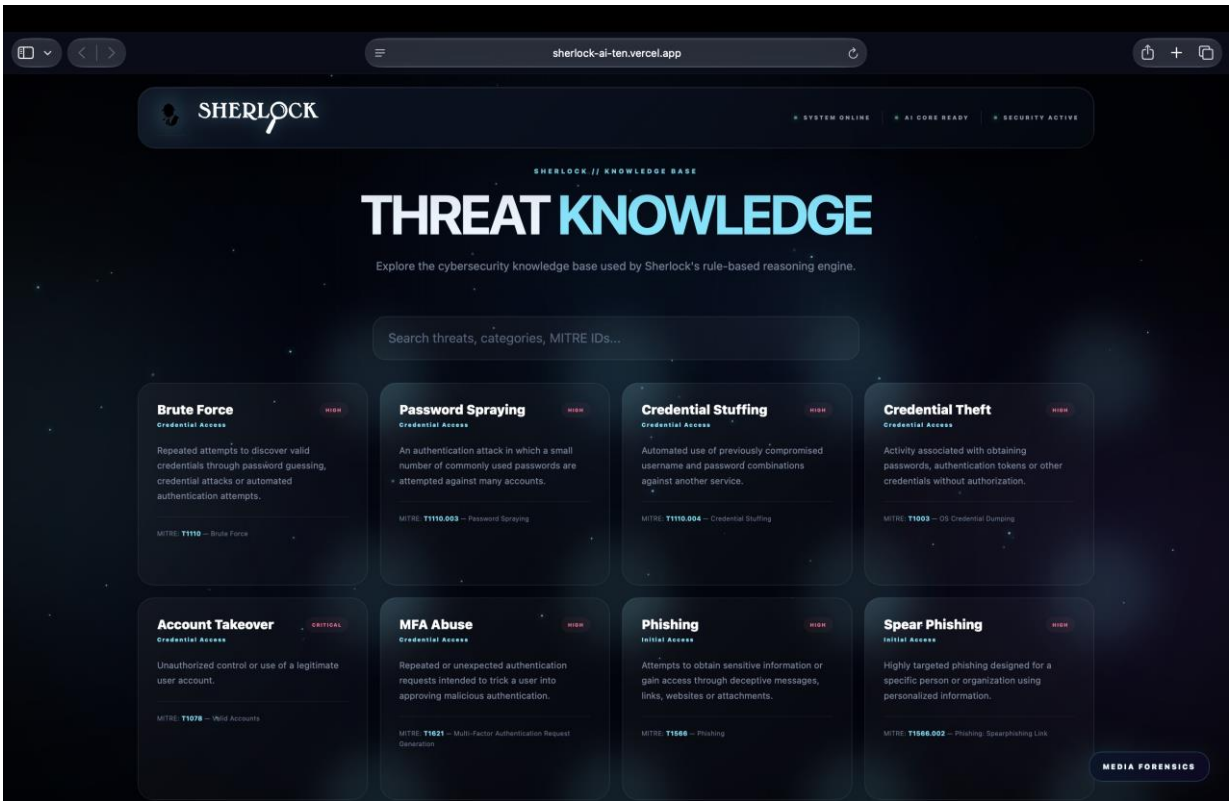


Figure S.3 SHERLOCK knowledge-base browser (illustrative mock-up)

The proposed knowledge-base view organizes rule identifiers, conditions, conclusions, severity criteria and recommendations. Administrative editing is a proposed capability unless verified in the application source.

APPLICATION SCREENSHOTS CONTINUED

The explanation view documents how the browser demonstration treats facts, rule matches and missing premises.

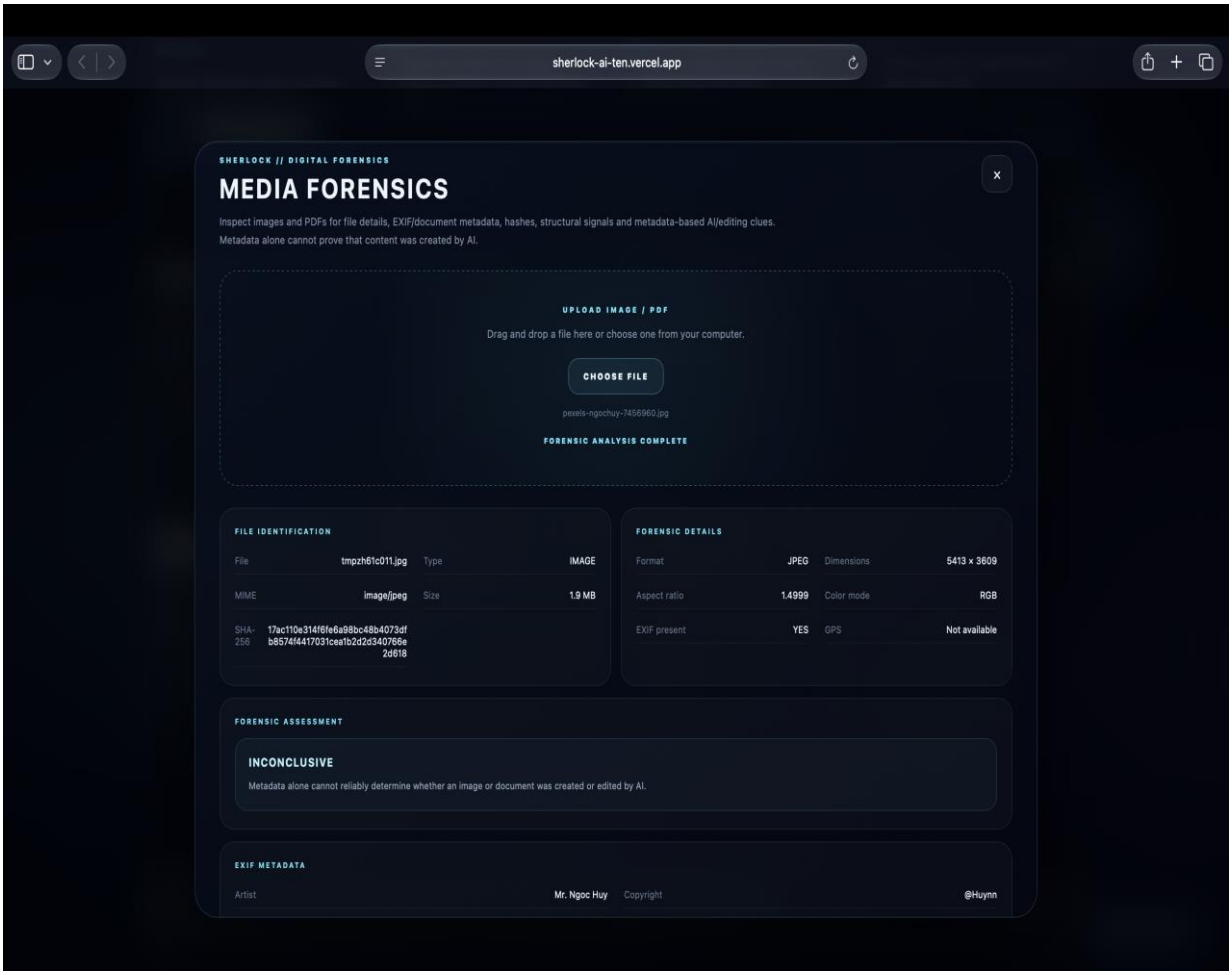


Figure S.4 SHERLOCK reasoning and limitations (illustrative mock-up)

This mock-up explains the intended media forensics tool , where the user puts his or her pictures and searches for metadata about those pictures

APPENDIX A

KNOWLEDGE BASE INVENTORY

Appendix A lists representative cybersecurity rules and indicators for the educational knowledge base. The inventory is illustrative and should be reconciled with the actual implemented rule definitions.

Table A.1 Threat inventory

No.	Rule ID / conclusion	Category
1	R-CRED-01: repeated failed logins exceed threshold in time window → possible brute force	Credential Access
2	R-CRED-02: repeated password attempts across accounts → possible password spraying	Credential Access
3	R-PHISH-01: unexpected link requests credentials → possible phishing	Phishing
4	R-PHISH-02: urgent request for one-time code → social-engineering warning	Social Engineering
5	R-ACCT-01: unrecognized successful login reported → account review	Account Misuse
6	R-ACCT-02: unexpected password reset notice → verify account activity	Account Misuse
7	R-MAL-01: unexpected executable download → potential malware indicator	Malware Indicators
8	R-MAL-02: unexplained file changes reported → investigate endpoint	Malware Indicators
9	R-DATA-01: unexpected public sharing → possible data exposure	Data Exposure
10	R-NET-01: unknown outbound connection reported → review network logs	Network Anomaly
11	R-SOC-01: message impersonates trusted authority and requests secret	Social Engineering
12	R-CRED-03: login failure count missing → ask for time/count context	Credential Access
13	R-PHISH-03: sender and destination cannot be verified → caution	Phishing
14	R-ACCT-03: MFA prompt not initiated by user → deny and report	Account Misuse
15	R-DATA-02: sensitive file sent to unintended recipient → contain exposure	Data Exposure

No.	Rule ID / conclusion	Category
16	R-MAL-03: security warning from installed protection tool → investigate	Malware Indicators
17	R-NET-02: unexpected connection follows suspicious attachment → investigate	Network Anomaly
18	R-GEN-01: evidence does not satisfy a known rule → insufficient evidence	General
19	R-GEN-02: contradictory facts for same account/time → request correction	General

The sample rules are design examples, not a verified count of implemented rules. The active rule inventory must be established from the submitted source code.

Use stable rule identifiers when maintaining the knowledge base. A display-label correction should not create a duplicate rule or break the link to an earlier saved analysis. Keep each published inventory tied to its version.

APPENDIX A

KNOWLEDGE BASE INVENTORY

Table A.2 Threat inventory continued

No.	Rule ID / conclusion	Category
20	R-CRED-04: known threshold and window satisfied → possible brute force	Credential Access
21	R-CRED-05: repeated login failures from multiple sources → investigate	Credential Access
22	R-PHISH-04: shortened or unexpected link → inspect safely without opening	Phishing
23	R-PHISH-05: attachment unexpected and sender unverified → do not open	Phishing
24	R-ACCT-04: account settings changed unexpectedly → review account	Account Misuse
25	R-ACCT-05: session alert not recognized → secure account	Account Misuse
26	R-MAL-04: unfamiliar application requests elevated access → verify	Malware Indicators
27	R-DATA-03: access permission broader than intended → restrict and review	Data Exposure
28	R-NET-03: unexpected service exposure reported → check configuration	Network Anomaly
29	R-SOC-02: request to bypass normal approval → verify independently	Social Engineering
30	R-GEN-03: no recognized facts → ask for additional details	General
31	R-GEN-04: evidence supports caution but not category → investigate	General
32	R-CRED-06: threshold not met → no brute-force conclusion	Credential Access
33	R-PHISH-06: known trusted link not established by facts → caution	Phishing
34	R-ACCT-06: unknown device report without corroboration → review	Account Misuse
35	R-DATA-04: suspected disclosure → preserve evidence and notify owner	Data Exposure
36	R-MAL-05: suspicious file cannot be verified → do not execute	Malware Indicators

No.	Rule ID / conclusion	Category
37	R-NET-04: anomaly lacks context → inspect logs before classifying	Network Anomaly
38	R-GEN-05: evidence conflict → correction required	General

The second inventory table continues the illustrative rules and their recommended actions. Rule coverage does not establish effectiveness against all real-world attacks.

Coverage and review status

Category labels support organization and browsing; they do not change the scoring formula. The loader should preserve a clear mapping from each threat profile to its six indicator weights and required-indicator set.

Any new rule should be structurally validated and assigned a review status before publication.

APPENDIX B

PROPOSED DATA DICTIONARY

Table B.1 Knowledge data dictionary

Table	Key fields	Constraints and notes
knowledge_rules	rule_id, premises, conclusion, category, severity_rule, recommendation_ids	Unique rule_id; validated structured conditions; no executable imports.
indicator_catalog	indicator_id, label, aliases, description	Unique canonical ID; aliases reviewed.
threat_categories	category_id, name, description	Stable identifiers and controlled vocabulary.
rule_versions	version_id, created_at, status, reviewer	Only reviewed versions may be published.
recommendations	recommendation_id, text, applicability	Defensive, proportionate and reviewed.
severity_criteria	criterion_id, condition, severity	Documented thresholds and rationale.

Table B.1 defines proposed knowledge records, rule premises, conclusions, severity criteria and recommendation mappings. Each rule should have a stable identifier and validation constraints.

Each rule should have a non-empty, well-formed set of premises and a valid conclusion. The loader should reject malformed conditions or unsupported severity values before evaluation.

Publication checks

Before a knowledge version becomes active, validate every referenced indicator, premise, conclusion and recommendation. All identifiers must resolve to known records so that the explanation can accurately describe the rule that fired.

A rejected draft should produce a reviewable error list and leave the last published snapshot available. Successful publication should identify the exact version used by subsequent incident analyses.

APPENDIX B

PROPOSED DATA DICTIONARY

Table B.2 Application data dictionary

Table	Key fields	Constraints and notes
users (optional)	user_id, password_hash, role	Only if authentication is implemented; never store plaintext passwords.
analysis_history (optional)	analysis_id, owner_id, knowledge_version, result_snapshot	Owner checks and explicit retention policy.
analysis_facts (optional)	analysis_id, indicator_id, state, context	Foreign keys; preserve unknown separately from absent.
audit_events (optional)	event_id, actor_id, action, timestamp	Avoid unnecessary sensitive incident content.
sessions (optional)	session_id, user_id, expiry	Secure, expiring session tokens.

Table B.2 describes the proposed application data fields for user input, analysis results and optional history. These fields are design suggestions, not proof that a database is implemented.

If SQLite persistence is implemented, foreign-key checks should be enabled for every connection. Administrative deletion should not silently remove information needed to interpret older results; deactivation or versioned replacement is preferable. Guest requests should not be stored by default.

Storage and retrieval checks

The history write should store the result and its observation rows atomically. Retrieval should check ownership before returning either the observation list or the snapshot, including when the request is made through an export route.

Retention and deletion behavior should be specified before personal histories are enabled. Tests should confirm that deleting one account's incident analysis does not alter shared knowledge records
or another account's data.

A saved result snapshot should retain the original facts, fired rules and explanation even when display labels or active knowledge rules change later.

APPENDIX C

INFERENCE PSEUDOCODE

C.1 Prototype scoring behavior

```
INPUT: user_description, structured_observations, knowledge_base

facts = normalize_and_validate(user_description, structured_observations)
results = []

FOR each rule R in knowledge_base:
   IF every premise of R is satisfied by facts:

      record matched premises, missing premises and rule identifier

      IF all required premises are satisfied:
         add the rule conclusion to derived facts

      append the conclusion, evidence trace and recommendation mapping

repeat rule evaluation until no new fact is derived or a defined limit is reached
```

This pseudocode describes the intended forward-chaining workflow. It is illustrative and must be checked against the implemented engine before being presented as executed source logic.

For R rules and F facts, a simple full-scan engine may require $O(R \times F)$ premise checks per pass; the total depends on the number of inference passes and rule structure.

Implementation notes

Keep the rounding order identical across the reusable engine and its regression examples. A required-indicator penalty changes the already rounded percentage in the prototype, so delaying all rounding until display can produce a different result.

The returned result should carry enough evidence to render the explanation without rerunning inference. This also supports a direct comparison between a saved snapshot and any exported representation.

APPENDIX C

INFERENCE PSEUDOCODE

C.2 Proposed controlled inference workflow

```
INPUT: validated incident facts and one knowledge-base version
reject malformed inputs and preserve unknown facts as unknown
evaluate rule premises; identify conflicts or missing evidence
IF no rule is supported: RETURN insufficient-evidence result
FOR each fired rule:
    record premises, conclusion, severity basis and recommendations
IF a newly derived fact may satisfy another rule:
    continue bounded forward chaining
DISPLAY supported findings, matched evidence and reasoning trace
IF history is implemented and the user explicitly requests saving:
    apply authorization and storage checks
```

This workflow is a design specification. It introduces explicit rule traces and a no-match path; actual implementation status must be confirmed from the source code.

A resolution demonstration should use finite clauses, a defined inference limit and a proof trace. Failure to find a refutation within a resource limit must be reported as inconclusive rather than false.

Workflow outcomes

Define separate result types for correction required, no match, ranked output and storage failure. Each type should identify the next available action so that the interface does not represent an incomplete request as a successful incident analysis.

If saving is implemented, the operation should store the result already computed against the current knowledge version. It should not silently recalculate against newly published rules while presenting the earlier explanation.

APPENDIX D

IMPLEMENTATION AND ACCEPTANCE PLAN

Table D.1 Planned acceptance tests

ID	Scenario	Acceptance condition
T01	Rule premises all satisfied	Expected rule fires and trace lists every satisfied premise.
T02	One required premise missing	Rule does not assert conclusion; missing evidence is displayed.
T03	Unknown indicator submitted	Unknown input is rejected or retained as unknown.
T04	Conflicting facts	System requests correction or explicitly reports conflict.
T05	No input	Insufficient-evidence response is shown.
T06	Duplicate observation	Duplicate does not create additional evidence.
T07	Severity threshold not met	High severity is not assigned without defined support.
T08	Rule definition malformed	Invalid rule is rejected before evaluation.
T09	Unauthorized history request	Access denied if history functionality exists.
T10	Recommendation mapping	Displayed actions correspond to the fired rule.

Table D.1 states planned application acceptance criteria, distinct from the reconstructed arithmetic examples in Chapter 4. These scenarios have not been reported as passed. Each executed test should record its input, expected output, actual result and tested software version.

Acceptance evidence record

For each executed scenario, record the test identifier, input, expected result, actual result and software version. Include the knowledge version whenever the scenario depends on scoring. A failed case should retain enough evidence to reproduce the problem.

Repeat related cases after a fix. A change to export formatting, for example, should still preserve the evidence checked by T08, while a change to publication must preserve the consistent incident analysis behavior checked by T09.

APPENDIX D

IMPLEMENTATION AND ACCEPTANCE PLAN

Table D.2 Proposed development milestones

Stage	Deliverable	Exit condition
1. Knowledge design	Reviewed indicators and IF–THEN rules.	Rule IDs, premises and conclusions documented.
2. Inference engine	Rule matching and bounded forward chaining.	Controlled examples return expected traces.
3. Web interface	Input and results views using Flask.	Validation and insufficient-evidence states work.
4. Explanation	Evidence, severity rationale and recommendations.	Displayed explanation matches inference trace.
5. Verification	Functional and security test record.	Actual outcomes recorded against identified build.
6. Final review	Documentation and reproducible setup.	Implemented features separated from proposed extensions.

Table D.2 orders development milestones by dependency and defines their exit conditions. Knowledge validation precedes integration so that interface work does not conceal reasoning defects. Authentication and ownership checks must be completed before personal incident analyses are stored.